

To simulate Producer Consumer Problem.

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int buffer[10], bufsize, in, out, produce, consume, choice = 0;
```

```
    in = 0;
```

```
    out = 0;
```

```
    bufsize = 10;
```

```
    while (choice != 3)
```

```
    {
```

```
        printf("\n1. Produce \t 2. Consume \t 3. Exit");
```

```
        printf("\nEnter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice)
```

```
        {
```

```
            case 1:
```

```
                if ((in + 1) % bufsize == out)
```

```
                    printf("\nBuffer is Full");
```

```
            else
```

```
            {
```

```
                printf("\nEnter the value: ");
```

```
                scanf("%d", &produce);
```

```
                buffer[in] = produce;
```

```
                in = (in + 1) % bufsize;
```

```
            }
```

```
break;
```

case 2:

```
if (in == out)
```

```
    printf("\nBuffer is Empty");
```

```
else
```

```
{
```

```
    consume = buffer[out];
```

```
    printf("\nThe consumed value is %d", consume);
```

```
    out = (out + 1) % bufsize;
```

```
}
```

```
break;
```

```
}
```

```
}
```

```
}
```

Output

```
1. Produce    2. Consume    3. Exit
Enter your choice: 1
```

```
Enter the value: 1222
```

```
1. Produce    2. Consume    3. Exit
Enter your choice: 1
```

```
Enter the value: 1111
```

```
1. Produce    2. Consume    3. Exit
Enter your choice: 2
```

```
The consumed value is 1222
```

```
1. Produce    2. Consume    3. Exit
Enter your choice: 2
```

```
The consumed value is 1111
```

```
1. Produce    2. Consume    3. Exit
```

To simulate Dinning Philosopher Problem

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int tph, philname[20], status[20], howhung, hu[20], cho;
```

```
void one();
```

```
void two();
```

```
int main() {
```

```
    int i;
```

```

printf("\n\nDINING PHILOSOPHER PROBLEM");

printf("\nEnter the total number of philosophers: ");

scanf("%d", &tph);


for (i = 0; i < tph; i++) {
    philname[i] = i + 1;
    status[i] = 1;
}


printf("How many philosophers are hungry: ");

scanf("%d", &howhung);


if (howhung == tph) {
    printf("\nAll are hungry...\nDeadlock stage will occur.");
    printf("\nExiting...\n");
    return 0;
} else {
    for (i = 0; i < howhung; i++) {
        printf("Enter philosopher %d position (0 to %d): ", (i + 1), tph - 1);
        scanf("%d", &hu[i]);
        status[hu[i]] = 2;
    }


    do {
        printf("\n1. One can eat at a time\n2. Two can eat at a time\n3. Exit\nEnter your choice:
");
        scanf("%d", &cho);


        switch (cho) {

```

```

        case 1:
            one();
            break;
        case 2:
            two();
            break;
        case 3:
            exit(0);
        default:
            printf("\nInvalid option. Please try again.");
    }
} while (1);
}

return 0;
}

void one() {
    int pos = 0, x, i;
    printf("\nAllowing one philosopher to eat at a time\n");

    for (i = 0; i < howhung; i++, pos++) {
        printf("\nP%d is granted to eat", philname[hu[pos]]);
        for (x = pos + 1; x < howhung; x++) {
            printf("\nP%d is waiting", philname[hu[x]]);
        }
    }
}

```

```

void two() {
    int i, j, s = 0, t, r, x;

    printf("\nAllowing two philosophers to eat at the same time\n");

    for (i = 0; i < howhung; i++) {
        for (j = i + 1; j < howhung; j++) {
            // Avoid adjacent philosophers and circular adjacency
            if (abs(hu[i] - hu[j]) >= 1 && abs(hu[i] - hu[j]) != (tph - 1)) {
                printf("\n\nCombination %d:", ++s);

                t = hu[i];
                r = hu[j];

                printf("\nP%d and P%d are granted to eat", philname[t], philname[r]);

                for (x = 0; x < howhung; x++) {
                    if (hu[x] != t && hu[x] != r) {
                        printf("\nP%d is waiting", philname[hu[x]]);
                    }
                }
            }
        }
    }
}

if (s == 0) {
    printf("\nNo valid pair of philosophers found to eat together.");
}
}

```

Output

DINING PHILOSOPHER PROBLEM

Enter the total number of philosophers: 5

How many philosophers are hungry: 3

Enter philosopher 1 position (0 to 4): 3

Enter philosopher 2 position (0 to 4): 4

Enter philosopher 3 position (0 to 4): 2

1. One can eat at a time

2. Two can eat at a time

3. Exit

Enter your choice: 1

Allowing one philosopher to eat at a time

P4 is granted to eat

P5 is waiting

P3 is waiting

P5 is granted to eat

To simulate reader writet Problem using Semaphore.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define MAX_READERS 5
```

```
sem_t rw_mutex;
```

```
sem_t mutex;

int reader_count = 0;
int read_batch = 0;

void* reader(void* arg) {
    int id = *((int*)arg);
    free(arg);

    sem_wait(&mutex);
    reader_count++;
    if (reader_count == 1) {
        sem_wait(&rw_mutex);
    }
    sem_post(&mutex);

    printf("Reader %d is reading...\n", id);
    sleep(1);

    sem_wait(&mutex);
    reader_count--;
    read_batch++;
    if (reader_count == 0) {
        sem_post(&rw_mutex);
    }
    if (read_batch == MAX_READERS) {
        printf("5 readers have read. Writer can write now.\n");
        read_batch = 0;
    }
    sem_post(&mutex);
}
```



```
    pthread_exit(NULL);  
}
```

```
void* writer(void* arg) {  
    int id = *((int*)arg);  
    free(arg);
```

```
    sem_wait(&rw_mutex);
```

```
    printf("Writer %d is writing...\n", id);  
    sleep(2);
```

```
    sem_post(&rw_mutex);  
    pthread_exit(NULL);  
}
```

```
int main() {  
    pthread_t r[10], w[2];  
  
    sem_init(&rw_mutex, 0, 1);  
    sem_init(&mutex, 0, 1);  
  
    for (int i = 0; i < 10; i++) {  
        int* id = malloc(sizeof(int));  
        *id = i + 1;  
        pthread_create(&r[i], NULL, reader, id);  
        sleep(1);  
    }
```

```
for (int i = 0; i < 2; i++) {  
    int* id = malloc(sizeof(int));  
    *id = i + 1;  
    pthread_create(&w[i], NULL, writer, id);  
    sleep(5);  
}
```

```
for (int i = 0; i < 10; i++) {  
    pthread_join(r[i], NULL);  
}
```

```
for (int i = 0; i < 2; i++) {  
    pthread_join(w[i], NULL);  
}
```

```
sem_destroy(&rw_mutex);  
sem_destroy(&mutex);
```

```
return 0;
```

Output

```
Reader 1 is reading...
Reader 2 is reading...
Reader 3 is reading...
Reader 4 is reading...
Reader 5 is reading...
5 readers have read. Writer can write now.
Reader 6 is reading...
Reader 7 is reading...
Reader 8 is reading...
Reader 9 is reading...
Reader 10 is reading...
5 readers have read. Writer can write now.
Writer 1 is writing...
Writer 2 is writing...
```

```
=== Code Execution Successful ===
```

```
}
```